

POO 3 (JAVA ET JAVA AVANCÉ)

Prof. Nisrine DAD

4° Ingénierie Informatique et Réseaux - Semestre I

Ecole Marocaine des Sciences d'Ingénieur

Année Universitaire : 2024/2025

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Premier exemple de programme Java

Fichier PremierTest.java

```
public class PremierTest
//La classe est l'unité de base de nos programmes.
{ //Accolade débutant la classe PremTest
    public static void main(String[] args)
    { //Accolade débutant la méthode main
        /* Pour l'instant juste une instruction. Ce commentaire s'étale sur
        plusieurs lignes */
        System.out.println("bonjour"); // Une instruction se termine par une ;
    } //Accolade fermant la méthode main
} //Accolade fermant la classe PremTest
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGUAGE JAVA

Exemple de programme Java

```
public class HelloEtudiant {  
    public static void main(String[] args) {  
        sayHelloTo("Amine Amini"); // sayHelloTo(args[0]) si on veut  
        passer le nom en paramètre lors de l'exécution  
    }  
    // affiche le message "hello" à l'étudiant fourni  
    private static void sayHelloTo(String nomEtudiant) {  
        System.out.println("Hello " + nomEtudiant);  
    }  
}
```

Cas de l'utilisation de args[o]

```
d:\EMSI>javac helloEtudiant.java  
  
d:\EMSI>java HelloEtudiant  
Hello Amine Amini
```

```
D:\EMSI\JavaFiles>javac HelloEtudiant.java  
  
D:\EMSI\JavaFiles>java HelloEtudiant "Amini Amine"  
Hello Amini Amine
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Contenu du byte-code

- Byte-code (HelloEtudiant.class) (ou code binaire – langage machine).

```

1  Èp°%NULNULNUL=NUL:
2  NUL$PXNULËTXBETNULEOTFFNULENQNULACK$OHNULDËjava/lang/Object$OHNULACK<init>$OHNULËTX()V
3  NULBSNUL    BETNUL
4  FFNULVFNULFFSOHNUL
5  HelloEtudiant$OHNUL
6  sayHelloTo$OHNULNAR(Ljava/lang/String;)V
7  NUL$ONUL$TBETNULDËFFNULDC1NULDC2SOHNUL$Tjava/lang/Float$OHNUL
8  parseFloat$OHNULNAR(Ljava/lang/String;)F    NULDC4NULNARBETNULSYNFFNULËTB3NULCAN$OHNULDË
9  java/lang/System$OHNULËTXout$OHNULNARL$java/io/PrintStream;
10 NUL$UBNULË$CBETNULË$FFNULGSNULËRSOHNULDC3java/io/PrintStream$OHNULBETprintln$OHNULEOT
    (F)VDC2NULNULNUL    FFNUL!NUL"SOHNULËTBmakeConcatWithConstants$OHNUL
    &(Ljava/lang/String;)Ljava/lang/String;
11 NUL$UBNULË$FFNULGSNULËFFSOHNULEOTCode$OHNUL$TLineNumberTable$OHNULEOTmain$OHNULSYN
    ([Ljava/lang/String;)V$OHNUL
12 SourceFile$OHNULDC2HelloEtudiant.java$OHNULDËBootstrapMethods$STACKNUL-
    NUL    NUL    BETNUL0FFNUL:    NUL1$OHNUL$java/lang/invoke/StringConcatFactory$OHNUL
    {Ljava/lang/invoke/MethodHandles$Lookup;Ljava/lang/String;Ljava/lang/invoke/MethodType;Ljava/lang/
    String;[Ljava/lang/Object;)Ljava/lang/invoke/CallSite;$BSNUL3$OHNULBETHello    $OHNULFF
    InnerClasses$BETNUL6$OHNUL$java/lang/invoke/MethodHandles$Lookup$BETNUL8$OHNULËRS
    java/lang/invoke/MethodHandles$OHNULACKLookupNUL
    !NULBSNUL$TXNULNULNULNULNULËTXNULNUL$OHNULENQNULACKNUL$OHNUL
    $NULNULNULGSNUL$OHNUL$OHNUL$OHNULNULNULËNQ*    NUL$OHNULNULNUL$OHNUL
    &NULNULNULACKNUL$OHNULNULNUL$PXNUL    NUL'NUL(NUL$OHNUL$NULNULNUL@NULËTXNUL$TXNULNULNULË$ËTX
    2,NULBET*ËOT2,NUL
    *ËNQ2,NUL
13 bD$NULDC3#1NULËM+NULNULNUL$OHNUL
    &NULNULNULDC2NULBOTNULNULNULËNQNULACKNULACKNULDC4NULBETNULË$CNULBSNUL
14 NULVFNULFFNUL$OHNUL$NULNULNUL    NUL$TXNUL$OHNULNULNUL
    $NULDC3*°NULGSNULNUL:NUL#±NULNULNUL$OHNUL&NULNULNUL
15 NUL$TXNULNULNULVFNULFFNULFFNULËTXNUL    NULNULNUL$TXNUL*NUL+NULNULNULBSNUL$OHNUL,NUL$OHNUL
16 2NUL4NULNULNUL
17 NUL$OHNUL5NUL7NUL9NULËM

```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Fonction main

- Le nom du fichier doit être le même que celui de la classe avec l'extension .java;
- Java est sensible à la casse des lettres (PremierTest # premierTest).
- Pour pouvoir faire un programme exécutable, il faut toujours une classe contenant une méthode particulière, la méthode « main ».
- C'est le point d'entrée dans le programme : le microprocesseur sait qu'il va commencer à exécuter les instructions à partir de cet endroit.
- Les principes du code propre exigent qu'aucune logique ne soit écrite à l'intérieur de la méthode `main` . Tout le travail doit être délégué à des fonctions bien nommées.

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Fonction main

- **public** : obligatoire pour que votre programme puisse s'exécuter.
- **Static**: précise que la méthode main de la classe *PremierTest* n'est pas liée à une instance (objet) particulière de la classe.
- **String[] args** : permet de récupérer des arguments transmis au programme au moment de son lancement. On peut lancer un programme sans fournir d'arguments, mais l'indication `String args[]` est obligatoire.
- **System.out.println ("Bonjour")** ; cette ligne sert à définir le contenu de notre programme qui va afficher le message Bonjour suivi d'un retour à la ligne suivante dans la fenêtre de console.

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Instructions, blocs, blancs et commentaires

- Les instructions Java se terminent par un ;
- Les blocs sont délimités par { pour le début de bloc } pour la fin du bloc.
- Un bloc permet de définir un regroupement d'instructions.
- La définition d'une classe ou d'une méthode se fait dans un bloc.
- Les espaces, tabulations, sauts de ligne sont autorisés. Cela permet de présenter un code plus lisible.
- La séquence // permet d'insérer un commentaire sur une seule ligne.
- La séquence /* permet d'insérer un commentaire sur plusieurs lignes. La séquence */ marque la fin du commentaire.

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

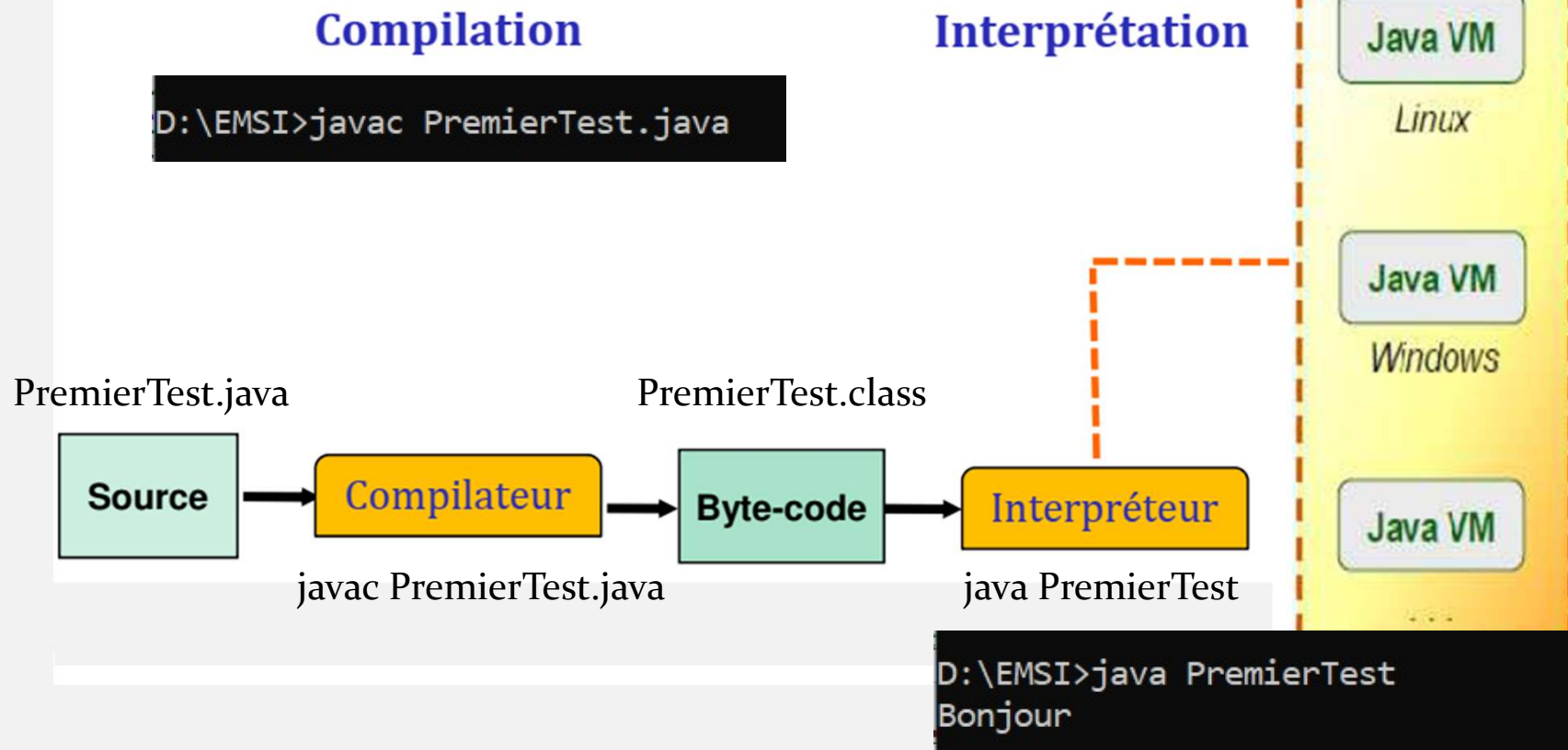
Compilation et exécution

- **Etape 1:** Compilation en bytecode java dans une console DOS:
javac PremierTest.java → Génère un fichier PremierTest.class
- **Etape 2:** Exécution du programme (toujours depuis la console DOS) sur la JVM :
java PremierTest → Affichage de “Bonjour” dans la console

```
D:\EMSI>javac PremierTest.java  
  
D:\EMSI>java PremierTest  
Bonjour
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Compilation et exécution



II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Compilation et exécution

- **Résumé:**
- `javac PremTest.java` :
 - Compilation en Byte-code java.
 - Indication des erreurs de syntaxe éventuelles.
 - Génération d'un fichier `PremierTest.class` si pas d'erreurs.
- `java PremierTest`
 - java est la machine virtuelle.
 - Exécution du Byte-code.
 - Nécessité de la méthode `main`, qui est le point d'entrée dans le programme.

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Exécution dans un IDE

- Le traitement et l'exécution des différents programmes Java seront effectués par le fameux IDE: Eclipse.
- Donc, on va utiliser désormais la dernière version d'Eclipse (Eclipse IDE for Java Developers), qu'on peut la télécharger de la page:

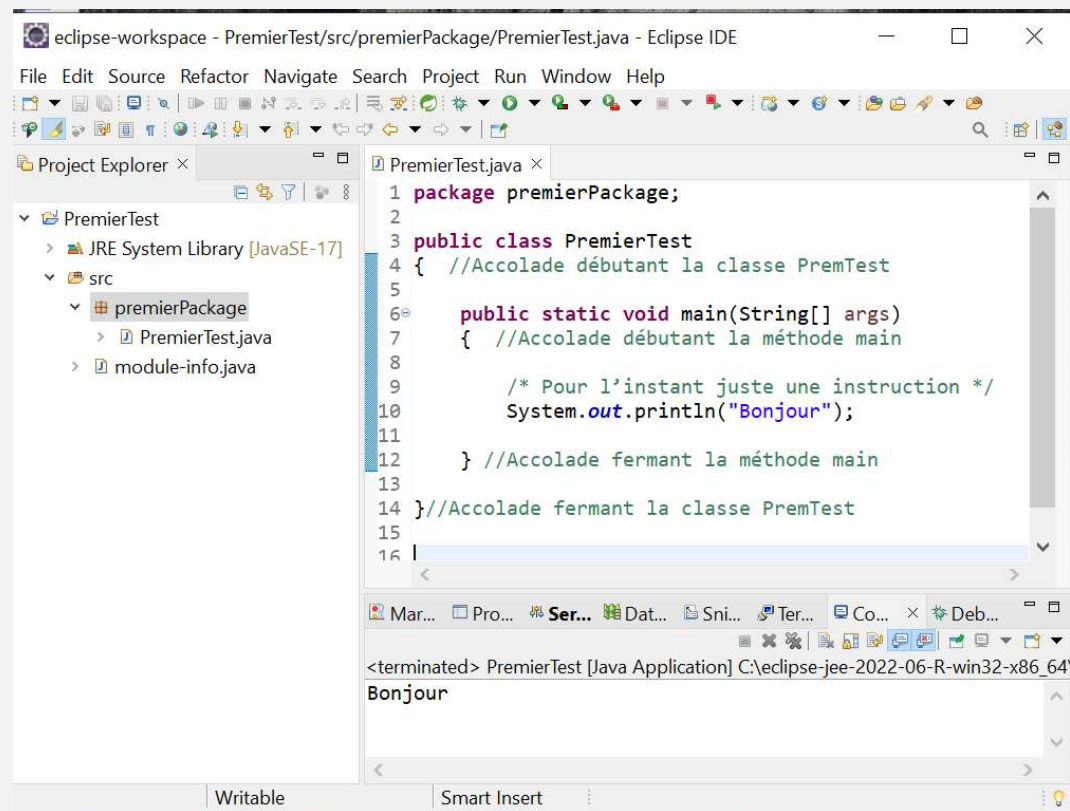
<http://www.eclipse.org/downloads>

- On peut télécharger la version de la JDK (8) sur le site suivant:

<http://www.oracle.com/technetwork/java/javase/downloads/index.html>

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Exécution dans un IDE



```
eclipse-workspace - PremierTest/src/premierPackage/PremierTest.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Project Explorer
PremierTest
  JRE System Library [JavaSE-17]
  src
    premierPackage
      PremierTest.java
      module-info.java
PremierTest.java
1 package premierPackage;
2
3 public class PremierTest
4 { //Accolade débutant la classe PremTest
5
6     public static void main(String[] args)
7     { //Accolade débutant la méthode main
8
9         /* Pour l'instant juste une instruction */
10        System.out.println("Bonjour");
11
12    } //Accolade fermant la méthode main
13
14 } //Accolade fermant la classe PremTest
15
16
<terminated> PremierTest [Java Application] C:\eclipse-jee-2022-06-R-win32-x86_64
Bonjour
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Portée des variables

- La portée (scope) d'une variable est la zone de code où elle a été déclarée.
- La portée variable générale s'applique à tous les blocs de code, y compris les classes.



Deux portées locales au sein d'une portée globale

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Portée des variables

- Un autre moyen nécessaire pour contrôler l'accès aux variables et aux fonctions est d'utiliser des niveaux de contrôle : public, protected, package-protected et private.
- **public** : visible pour tous et par conséquent le moins restrictif ;
- **protected (protégé)** : visible pour le package et l'ensemble de ses sous-classes ;
- **package-protected (protégé par paquet)** : généralement visible uniquement par le package dans lequel il se trouve (**paramètres par défaut**). Ne pas mettre de mot clé déclenche ce niveau de contrôle ;
- **private (privé)** : accessible uniquement dans le contexte dans lequel les variables sont définies (à l'intérieur de la classe dans laquelle il est situé).

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Portée des variables: Exercice

```
public class Rectangle {  
    public int width=4;  
    private int borderWidth=1;
```

```
    public static void main(String[] args) {  
        int mainWidth=28;
```

```
    }  
    public void printValues() {  
        final int newWidth=12;  
        if (true) {  
            int pixelSize=5;
```

```
        }  
        //Quelle variable pourrait être utilisée ici ?  
        System.out.println(?);
```

```
    }
```

```
}
```

- La variable **newWidth** est **accessible** car elle se trouve dans la même fonction.
- Les variables **width** et **borderWidth** sont aussi **accessibles** car elles se trouvent dans la classe Rectangle.
- La variable **pixelSize** **n'est pas accessible** car elle déclarée dans le bloc if.

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGUAGE JAVA

Lecture d'informations au clavier

- Pour la saisie de données au clavier, on va utiliser la classe **Scanner** (java.util.Scanner) qui regroupe plusieurs méthodes pratiques :
- nextInt(), nextDouble(), nextFloat(), nextLong(), nextLine(), ...

```
import java.util.Scanner;
public class ScannerIntTest{
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);//System.in: Entrée Standard

        System.out.println("Entrer un entier:");
        int a=sc.nextInt();
        System.out.println(" le nombre saisi est : " + a);
    }
}
```

```
D:\EMSI\JavaFiles>java ScannerIntTest
Entrer un entier:
6
le nombre saisi est :6
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Lecture d'informations au clavier

- **Remarque:** si vous avez appelé `nextInt()`, `nextDouble()` et que vous appelez directement après `nextLine()`, celle-ci ne vous invite pas à saisir une chaîne de caractères : elle vide la ligne commencée par les autres instructions.
- En effet, celles-ci ne repositionnent pas la tête de lecture, l'instruction `nextLine()` le fait à leur place.

```
import java.util.Scanner;
public class ScannerLineTest {
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.println("Saisissez un entier : ");
        int i = sc.nextInt();
        System.out.println("Saisissez une ligne : ");
        String str = sc.nextLine();
        System.out.println("FIN ! "); } }
```

```
D:\EMSI\JavaFiles>java ScannerLineTest
Saisissez un entier :
5
Saisissez une ligne :
FIN !
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Lecture d'informations au clavier

- Le programme précédent ne vous demande pas de saisir une chaîne et affiche directement « Fin ». Pour pallier ce problème, il suffit de vider la ligne après les instructions ne le faisant pas automatiquement :

```
import java.util.Scanner;
public class ScannerLineTest {
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.println("Saisissez un entier : ");
        int i = sc.nextInt();
        System.out.println("Saisissez une ligne : ");
        //On vide la ligne avant d'en lire une autre
        sc.nextLine();
        String str = sc.nextLine();
        System.out.println("FIN ! "); } }
```

```
D:\EMSI\JavaFiles>java ScannerLineTest
Saisissez un entier :
6
Saisissez une ligne :
Nouvelle Ligne
FIN !
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Lecture d'informations au clavier

Résumé:

- L'affichage des données se fait par *System.out.println ()*.
- La lecture des entrées clavier se fait via l'objet *Scanner*.
- Scanner se trouve dans le package *java.util* qu'il faut importer.
- Pour pouvoir récupérer ce que vous allez taper dans la console, vous devrez initialiser l'objet Scanner avec l'entrée standard, *System.in*.
- Il y a une méthode de récupération de données pour chaque type (sauf les char) : *nextLine()* pour les String, *nextInt()* pour les int ...

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Identificateurs

- **Attention** : Java distingue les majuscules des minuscules
- Pas de caractères spéciaux, pas d'espaces.
- **Conventions sur les identificateurs** :
- Si plusieurs mots sont accolés, utilisez le **Camel Case**.
- D'autre part suivez ces directives:
 - La première lettre est majuscule pour les classes et les interfaces. exemples : **MaClasse**, **MonInterface**.
 - La première lettre est minuscule pour les packages, les méthodes, les attributs et les variables. exemples : `setLongueur`, `i`, `unAttribut`.
 - Les constantes sont entièrement en majuscules. exemple : `LONGUEUR_MAX`

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGUAGE JAVA

Les mots réservés de Java

- Les mots réservés du langage Java ne peuvent pas être utilisés comme identifiant.

<code>abstract</code>	<code>default</code>	<code>goto</code>	<code>null</code>	<code>synchronized</code>
<code>boolean</code>	<code>do</code>	<code>if</code>	<code>package</code>	<code>this</code>
<code>break</code>	<code>double</code>	<code>implements</code>	<code>private</code>	<code>throw</code>
<code>byte</code>	<code>else</code>	<code>import</code>	<code>protected</code>	<code>throws</code>
<code>case</code>	<code>extends</code>	<code>instanceof</code>	<code>public</code>	<code>transient</code>
<code>catch</code>	<code>false</code>	<code>int</code>	<code>return</code>	<code>true</code>
<code>char</code>	<code>final</code>	<code>interface</code>	<code>short</code>	<code>try</code>
<code>class</code>	<code>finally</code>	<code>long</code>	<code>static</code>	<code>void</code>
<code>continue</code>	<code>float</code>	<code>native</code>	<code>super</code>	<code>volatile</code>
<code>const</code>	<code>for</code>	<code>new</code>	<code>switch</code>	<code>while</code>

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les types de base

TYPE PRIMITIF	TYPE "OBJET" ASSOCIÉ	PLACE EN MÉMOIRE	FOURCHETTE DE VALEURS (MIN & MAX)
boolean	Boolean	1 octet	true (1) false (0)
byte	Byte	1 octet	-128 à 127
char	Character	2 octets	Unicode (65536 caractères disponibles)
short	Short	2 octets	-32768 à 32767
int	Integer	4 octets	-2 147 483 648 à 2 147 483 647
long	Long	8 octets	-2^{63} à $+2^{63} - 1$
float	Float	4 octets	1.4×10^{-45} à 3.4×10^{38}
double	Double	8 octets	4.9×10^{-324} à 1.7×10^{308}

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les types de base

- On peut utiliser _ comme séparateur de milliers.
- On peut utiliser les différentes bases mathématiques.
- Exemple:

```
int varSeparateur=12_000;  
System.out.println("Variable avec séparateur: "+varSeparateur);  
int varDecimale=987; // Base 10: 0-> 9  
int varBinaire=0b101; // Base 2: 0-> 1  
int varOctale=0765; // Base 8: 0-> 7  
int varHexadecimale=0xF65; // Base 16: 0-> F  
System.out.println("Variable décimale: "+varDecimale+  
    "\nVariable décimale: "+varBinaire+  
    "\nVariable octale: "+varOctale+  
    "\nVariable hexadécimale: "+varHexadecimale);
```

```
Variable avec séparateur:  
12000  
Variable décimale: 987  
Variable décimale: 5  
Variable octale: 501  
Variable hexadécimale: 3941
```


II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les types de base

- Exemple (affichage binaire, octal et hexadécimal):

```
int varDecimale=987; // Base 10: 0-> 9
int varBinaire=0b101; // Base 2: 0-> 1
int varOctale=0765; // Base 8: 0-> 7
int varHexadecimale=0xF65; // Base 16: 0-> F
System.out.println("Variable décimale: "+varDecimale+
    "\nVariable binaire: "+Integer.toBinaryString(varBinaire)+
    "\nVariable octale: "+Integer.toOctalString(varOctale)+
    "\nVariable hexadécimale: "+Integer.toHexString(varHexadecimale));
```

Variable décimale: 987
Variable binaire: 101
Variable octale: 765
Variable hexadécimale: f65

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les opérateurs

	Symbole	Description	Exemple
Opérateurs arithmétiques	-	<i>soustraction</i>	$x-y$
	*	<i>multiplication</i>	$3 * x$
	/	<i>division</i>	$4/2$
	%	<i>modulo (reste de la division)</i>	$5\%2$
Opérateurs d'affectation	=	<i>Affectation</i>	$x=2$
	--	<i>Soustraction et affectation</i>	$x-=2$
	+=	<i>Addition et affectation</i>	$x+=2$

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les opérateurs

	Symbole	Description	Exemple
Opérateurs d'incrémentations et décrémentations	<code>++</code>	<i>Pré-incrémentation</i>	<code>++x</code>
	<code>++</code>	<i>Post-incrémentation</i>	<code>x++</code>
	<code>--</code>	<i>Pré-décrémentation</i>	<code>--x</code>
	<code>--</code>	<i>Post-décrémentation</i>	<code>x--</code>
Opérateurs relationnels	<code>==</code>	<i>égal à</i>	<code>x==2</code>
	<code><</code>	<i>inférieur à</i>	<code>x<2</code>
	<code><=</code>	<i>inférieur ou égal à</i>	<code>x<=3</code>
	<code>></code>	<i>supérieure à</i>	<code>x>2</code>
	<code>>=</code>	<i>supérieur ou égal à</i>	<code>x>=3</code>
	<code>!=</code>	<i>différent de</i>	<code>a != b</code>

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les opérateurs

	Symbole	Description	Exemple
Opérateurs logiques	<code>&&</code>	<i>et</i>	<code>a && b</code>
	<code>//</code>	<i>ou</i>	<code>a // b</code>
	<code>!</code>	<i>non</i>	<code>!a</code>
Opérateurs relationnels	<code>&</code>	<i>et</i>	<code>a & b</code>
	<code> </code>	<i>ou</i>	<code>a b</code>
	<code>^</code>	<i>ou exclusif</i>	<code>a ^ b</code>
	<code>~</code>	<i>non</i>	<code>~x</code>
	<code><<</code>	<i>décalage à gauche</i>	<code>a << 3</code>
	<code>>></code>	<i>décalage à droite</i>	<code>b >> 2</code>

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les structures de contrôle

- ***for***: Exécute un ensemble d'instructions un nombre de fois fixe, basé sur les valeurs limites inférieure et supérieure de l'énumérateur.

```
for (initialisation; condition ; incrémentation ou décrémentation) {  
    lesInstruction;  
}
```

- ***break***: Permet de terminer l'exécution d'une boucle (interrompre une séquence d'exécution).
- ***continue***: Permet l'interruption d'une itération en cours et retourne au début de la boucle avec exécution de la partie incrémentation (Ignorez quelques instructions à l'intérieur d'une boucle)

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGUAGE JAVA

```
for ( int i=0; i <10; i++) { // déclarations exécutées à chaque itération
    if(i == 2 || i == 5) {        continue; }
    System.out.println("Valeur de i : " + i + ".");
}
```

```
int [] myArray = { 10, 20, 30, 40, 50, 60, 70, 80, 90, 100 };
for (int i =0; i<myArray.length;i++) {
    if (myArray[i] == 50) {
        System.out.println ("J'ai trouvé ce que je cherche: "
            +myArray[i]+ " !");
        break; }
    System.out.println ("Je cherche encore. Je suis arrivé à "+myArray[i]);
}
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les structures de contrôle

- **while** : Exécute un ensemble d'instructions jusqu'à ce qu'une condition définie soit remplie.

```
while (expression booléenne) {  
    instructions;  
}
```

```
int nombreArbres = 0;  
while (nombreArbres < 10) {  
    nombreArbres += 1;  
    System.out.println("J'ai " + nombreArbres +  
                        " arbres");  
}
```

```
J'ai 1 arbres  
J'ai 2 arbres  
J'ai 3 arbres  
J'ai 4 arbres  
J'ai 5 arbres  
J'ai 6 arbres  
J'ai 7 arbres  
J'ai 8 arbres  
J'ai 9 arbres  
J'ai 10 arbres
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGUAGE JAVA

Les structures de contrôle

- **do, while**: Elle est très similaire à **while**, mais la condition est placée à la fin du bloc de code correspondant. De cette façon, le bloc de code sera toujours exécuté au moins une fois.

```
do {  
    instructions;  
} while (condition);
```

```
int pushUpGoal = 10;  
do{  
    System.out.println ("Push up " + pushUpGoal);  
    pushUpGoal -= 1;  
} while(pushUpGoal > 0);
```

```
Push up 10  
Push up 9  
Push up 8  
Push up 7  
Push up 6  
Push up 5  
Push up 4  
Push up 3  
Push up 2  
Push up 1
```


II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGUAGE JAVA

Les structures de contrôle

- *If, else if, else*

```
if (expression logique){  
    instructions;  
}  
  
else if (expression logique){  
    instructions;  
}  
  
else{  
    instructions;  
}
```

```
if (args.length==1) {  
    sayHelloTo(args[0]);  
}  
else if (args.length==2) {  
    sayHelloTo(args[0] + "-" + args[1]);  
}  
else if (args.length==3) {  
    sayHelloTo(args[0] + "-" + args[1] +  
        "-" + args[2]);  
}  
else {  
    sayHelloTo("world");  
}
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les structures de contrôle

- *switch, case, break, default*: Switch utilise ce qu'on appelle des cas, pour comparer une valeur et décider si un bloc de code associé doit être exécuté.

```
switch(variable) {  
    case Valeur 1: instruction 1;  
    break;  
    case ...:  
    break;  
    case Valeur n: instruction n;  
    break;  
    default: instruction;  
}
```

```
switch(args.length) {  
    case 0: sayHelloTo("world");  
    break;  
    case 1: sayHelloTo(args[0]);  
    break;  
    case 2: sayHelloTo(args[0] + "-" +  
        args[1]);  
    break;  
    default: System.out.println(« Vous  
        devez fournir au plus 2 arguments! »);  
}
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les structures de contrôle

Exercice: Quel est le résultat du programme suivant?

```
int n = 0;
do {
    if (n % 2 == 0) {
        System.out.println(n + " est pair");
        n += 3;
        continue;}

    if (n % 3 == 0) {
        System.out.println(n + " est multiple de 3");
        n += 5;}

    if (n % 5 == 0) {
        System.out.println(n + " est multiple de 5");
        break;}

    n += 1;
} while (true);
```

0 est pair
3 est multiple de 3
9 est multiple de 3
15 est multiple de 3
20 est multiple de 5

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGUAGE JAVA

Les structures de contrôle

Exercice: Quel est le résultat du programme suivant?

```
int n, p;  
  
n = p = 0;  
while (n < 5) n += 2; p++;  
System.out.println("A : n = " + n + ", p = " + p);  
  
n = p = 0;  
while (n < 5) { n += 2; p++;}  
System.out.println("B : n = " + n + ", p = " + p);
```

A : n = 6, p = 1
B : n = 6, p = 3

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les tableaux

- Les tableaux Java sont des structures pouvant contenir un nombre fixe d'éléments de même nature. Il peut s'agir d'objets ou de primitives.

Tableaux unidimensionnels

- **Déclaration** : `type nomTableau []` ; ou `type [] nomTableau` ;
- **Création**: `nomTableau = new type [dimension]`;

```
int[] tableauInt; // Déclaration d'un tableau de type int
Eleve tableauInt []; // Déclaration d'un tableau de type Eleve
String[] tableauString=new String[2]; // Déclaration et création d'un tableau de String
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les tableaux

- **Initialisation automatique:** Chaque élément du tableau est initialisé selon son type par l'instruction **new**:
- 0 pour les numériques, \0 pour les caractères, false pour les booléens null pour les chaînes de caractères et les autres objets.
- **Initialisation au moment de la déclaration:** `int [] x={1, 2, 3, 4, 5} ;//` tableau de cinq éléments de type int initialisé par{1, 2, 3, 4, 5}

```
String[] tableauString=new String[2]; /* initialisation automatique grâce à  
new */  
int[] tableauInt ={1,2,3,4,5}; // Initialisation au moment de la déclaration
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les tableaux

- Affectation des tableaux:

```
int [] t1 = new int[3] ;
```

```
for (int i=0 ; i<3 ; i++)
```

```
    t1[i] = i ;
```

```
int [] t2 = new int[2] ;
```

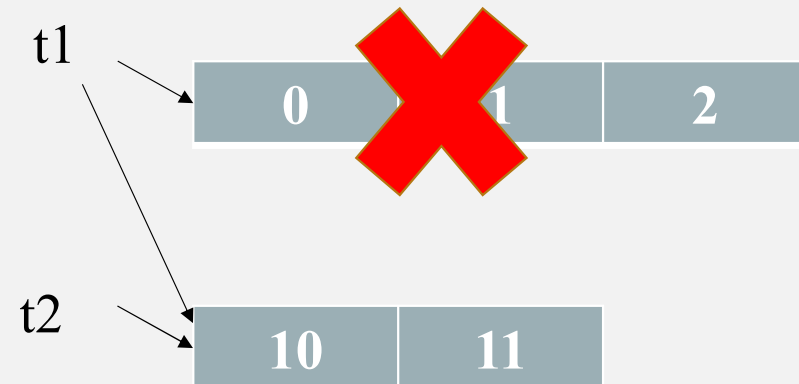
```
for (int i=0 ; i<2 ; i++)
```

```
    t2[i] = 10 + i ;
```

```
t1=t2;
```

```
t1[0] = 5; // Quel serait le contenu de t2[0]?
```

```
• System.out.println(t2[0]); // La valeur 5 est affichée
```



II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les tableaux

- **Taille d'un tableau:** Un tableau est un objet possédant l'attribut `length` : c'est la taille du tableau.

```
int t[] = new int[5] ;  
System.out.println ("taille de t : " + t.length) ; // affiche 5  
t = new int[3] ;  
System.out.println ("taille de t : " + t.length) ; // affiche 3
```

- **Utilisation d'un tableau:**
- **Exemple:** Un programme qui utilise un tableau de réels pour déterminer le nombre d'élèves d'une classe ayant une note supérieure à la moyenne de la classe.

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les tableaux

```
public class Moyenne{
    public static void main (String args[]) {
        int i, nbEl, nbElSupMoy ; double somme ; double moyenne ;
        Scanner sc=new Scanner (System.in);
        System.out.print ("Combien d'élèves ") ;
        nbEl = sc.nextInt();
        double notes[] = new double[nbEl] ;
        for (i=0 ; i<nbEl ; i++){
            System.out.print ("donnez la note numéro " + (i+1) + " : " ) ;
            notes[i] = sc.nextDouble() ;}
        for (i=0, somme=0 ; i<nbEl ; i++) {
            somme += notes[i] ;}
        moyenne = somme / nbEl ;
        System.out.println ("\n moyenne de la classe " + moyenne) ;
        for (i=0, nbElSupMoy=0 ; i<nbEl ; i++ )
            if (notes[i] > moyenne)
                nbElSupMoy++ ;
        System.out.println (nbElSupMoy + " élèves ont plus de cette moyenne") ;
    }
}
```

```
Combien d'élèves 5
donnez la note numéro 1 : 11
donnez la note numéro 2 : 14
donnez la note numéro 3 : 13
donnez la note numéro 4 : 10
donnez la note numéro 5 : 16
moyenne de la classe 12.8
3 élèves ont plus de cette moyenne
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGUAGE JAVA

Les tableaux

Tableaux multidimensionnels

- **Déclaration** : `type nomTableau [][]` ; ou `type [][] nomTableau` ;
- **Création**: `nomTableau = new type [n1][n2]`;

```
// Déclaration et création d'un tableau à 2 dimensions de String  
String[][] tableauString=new String[5][10];
```

- Un tableau à 2 dimensions peut être initialisé comme suit:

```
int t21[][] = {{1, 2, 3}, {4, 5, 6}} // {{ligne 1},{ligne 2}}
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les tableaux

Tableaux multidimensionnels

- **Déclaration et création:** Toutes les lignes d'un tableau à 2 dimensions n'ont pas forcément le même nombre d'éléments :

```
int t22[][] ;
t22 = new int[5][];
for( int i = 0; i< t22.length; i++){
    t22[i]= new int [i+1]; }

for( int i = 0; i< t22.length; i++){
    for( int j = 0; j<t22[i].length; j++){
        //accès à t22[i][j] pour le remplir par exemple }
    }
}
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les fonctions

- Une **fonction** peut être considérée comme un bloc de code avec un nom, qui exécute un service.
- Lorsqu'une fonction est située à l'intérieur d'une classe, elle s'appelle une **méthode**.
- Tout le code Java doit être situé à l'intérieur de classes.
- Les classes sont contenues dans des packages.
- Lorsqu'on lance le programme, c'est la fonction **main()** qui se lance.
- La fonction **main()** est le morceau de code que l'interpréteur Java recherche lorsque vous démarrez un programme.

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les fonctions

- Si on écrit toute la logique de notre programme à l'intérieur du `main`, cela pourrait donner une trop grande quantité de code à un seul endroit.
- Ce serait difficilement compréhensible pour nous, les humains, et compliquerait la maintenance de votre programme.
- C'est la raison pour laquelle on doit organiser notre code en classes.
- Dans les bonnes pratiques, la méthode `main` doit être la plus courte possible, en appelant uniquement les méthodes nécessaires.

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les fonctions prédéfinies

- Les classes prédéfinies en java offrent plusieurs méthodes.
- Il faut utiliser le site web Javadoc pour trouver toutes les méthodes d'une classe.
- Cela permet de gagner du temps par la suite.
- Par exemple, **String** est une classe qui définit un état et un comportement :
 - Son état est la chaîne de caractères que vous stockez. La valeur réelle est définie pour chaque objet lorsque vous l'instanciez.
 - Son comportement est l'ensemble des méthodes que la classe `String` définit, et qui vous permettent d'opérer sur la chaîne que vous stockez.

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les fonctions prédéfinies

- Exemple:

```
String exemple="hello";  
exemple=exemple.toUpperCase();
```

String	replace (char oldChar, char newChar)	Returns a string resulting from replacing all occurrences of oldChar in this string with newChar.
String	toUpperCase ()	Converts all of the characters in this String to upper case using the rules of the default locale.
boolean	contains (CharSequence s)	Returns true if and only if this string contains the specified sequence of char values.

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGUAGE JAVA

Les fonctions

- Les fonctions sont considérées comme des modules qui permettent d'éviter de tout rassembler dans un seul et même bloc de code, afin **d'améliorer la lisibilité** de son programme.
- Cela permet aussi de diviser le programme en sous parties, qui pourront être ensuite **réutilisées** dans d'autres programmes.

```
public static <type_de_la_variable_de_retour> <nom_de_la_fonction> (<paramètre1>,  
<paramètre2>, ...) {  
    // Instructions ici  
}
```


II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGUAGE JAVA

Les fonctions

Exemple:

```
public class Test {  
    public static int MultiplierParDeux(int entree) {  
        int sortie; sortie = entree * 2;  
        return sortie; }  
  
    public static void main(String args[]) {  
        int x = 3;  
        int y = MultiplierParDeux(x);  
        System.out.println(y); }  
}
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les fonctions

- Les principes du code propre exigent qu'aucune logique ne soit écrite à l'intérieur de la méthode `main` .
- Tout le travail doit être délégué à des fonctions bien nommées.

```
public class HelloEtudiant {  
    public static void main(String[] args) {  
        sayHelloTo("Amine Amini");  
        // affiche le message "hello" à l'étudiant fourni  
    private static void sayHelloTo(String nomEtudiant) {  
        System.out.println("Hello " + nomEtudiant);  
    }  
}
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les fonctions

- Les fonctions peuvent avoir des **paramètres** et des **valeurs de retour** (return).
- Une **valeur de retour** est le résultat de l'exécution de la fonction. Elle peut être renvoyée au bloc de code qui a appelé la fonction, et être utilisée si besoin est.
- **Les paramètres** sont les entrées d'une fonction nécessaires à l'exécution et à l'obtention d'un résultat.
- **Les paramètres** sont des variables définies par leur nom et par leur type. Ces paramètres sont spécifiés dans la déclaration de la fonction.
- Lorsque on **appelle** une fonction, on fait passer des valeurs à ces variables.
- Ces valeurs sont appelées **arguments**.

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Les fonctions mathématiques

- **Math.pow ()** : x puissance a
- **Math.sqrt()** : détermine la racine carrée d'un nombre
- **Math.abs ()** : $|x|$ (valeur absolue de x)
- **Math.min ()** : détermine le minimum de deux nombres
- **Math.max()** : détermine le maximum de deux nombres
- **Math.random()** : donne un nombre au hasard entre 0 et 1
- **Math.sin ()** : calcule le sinus d'un angle.
- **Math.cos ()** : calcule le cosinus d'un angle.
- **Math.tan ()** : calcule la tangente d'un angle.
- **Math.exp ()** : calculer l'exponentielle d'un nombre.
- **Math.log ()** : calcule le logarithme d'une valeur.
- **Math.floor()** : arrondit à l'entier inférieur

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Exercice: tableaux et fonctions

Exemple d'exécution du programme:

3 7 1 9 2 8 9 1 2 5

Il y a 2 éléments de valeur 9 dans le tableau

Il y a 7 éléments plus grand que 4 ou plus petit que 2

La somme des éléments est 47

La moyenne arithmétique des éléments est 4.7

Le plus grand élément vaut: 9

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGUAGE JAVA

Exercice: tableaux et fonctions

Operations sur les matrices:

1. Ecrire une fonction qui teste si une matrice est carrée.

```
public static boolean matriceCarree(int[][] matrice) {  
    boolean resultat;  
    if (matrice.length==matrice[0].length)  
        resultat=true;  
    else  
        resultat=false;  
    return resultat;  
}
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Exercice: tableaux et fonctions

Operations sur les matrices:

2. Ecrire une fonction qui test si 2 matrices ont la même taille.

```
public static boolean memeTaille(int[][] matrice1, int[][] matrice2 ) {  
    boolean resultat;  
    if (matrice1.length==matrice2.length &&  
        matrice1[0].length==matrice2[0].length)  
        resultat=true;  
    else  
        resultat=false;  
    return resultat;  
}
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Exercice: tableaux et fonctions

Operations sur les matrices:

3. Ecrire une fonction qui affiche une matrice. Utiliser la boucle for améliorée.

```
public static void afficherMatrice(int[][] matrice) {  
    for(int[] row:matrice) {  
        for(int element:row)  
            System.out.print(element+" ");  
        System.out.println();  
    }  
}
```


II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGUAGE JAVA

Exercice: tableaux et fonctions

Operations sur les matrices:

4. Ecrire une fonction qui calcule la transposée d'une matrice.

```
public static int[][] transpose(int[][] matrice) {  
    int rows=matrice.length;  
    int columns=matrice[0].length;  
    int[][] resultat=new int[columns][rows];  
    for(int i=0; i<columns;i++)  
        for(int j=0; j<rows;j++)  
            resultat[i][j]=matrice[j][i];  
    return resultat;  
}
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Exercice: tableaux et fonctions

Operations sur les matrices:

5. Ecrire une fonction qui calcule l'addition de 2 matrices.

```
public static int[][] addition(int[][] matrice1, int[][] matrice2) {  
    int rows=matrice1.length;  
    int columns=matrice1[0].length;  
    int[][] resultat=new int[rows][columns];  
  
    for(int i=0; i<rows;i++)  
        for(int j=0; j<columns;j++)  
            resultat[i][j]=matrice1[i][j]+matrice2[i][j];  
  
    return resultat;  
}
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Exercice: tableaux et fonctions

Operations sur les matrices:

6. Ecrire une fonction qui calcule la multiplication de 2 matrices.

```
public static int[][] multiplication(int[][] matrice1, int[][] matrice2) {  
    int rowsM1=matrice1.length;  
    int columnsM1=matrice1[0].length;  
    int columnsM2=matrice1[0].length;  
    int[][] resultat=new int[rowsM1][columnsM2];  
  
    for(int i=0; i<rowsM1;i++)  
        for(int j=0; j<columnsM2;j++)  
            for(int k=0; k<columnsM1;k++)  
                resultat[i][j]=resultat[i][j]+matrice1[i][k]*matrice2[k][j];  
    return resultat;  
}
```

$$\begin{matrix} 4 \times 2 \text{ matrix} \\ \begin{bmatrix} a_{11} & a_{12} \\ \cdot & \cdot \\ a_{31} & a_{32} \\ \cdot & \cdot \end{bmatrix} \end{matrix} \begin{matrix} 2 \times 3 \text{ matrix} \\ \begin{bmatrix} \cdot & b_{12} & b_{13} \\ \cdot & b_{22} & b_{23} \end{bmatrix} \end{matrix} = \begin{matrix} 4 \times 3 \text{ matrix} \\ \begin{bmatrix} \cdot & x_{12} & x_{13} \\ \cdot & \cdot & \cdot \\ \cdot & x_{32} & x_{33} \\ \cdot & \cdot & \cdot \end{bmatrix} \end{matrix}$$

$$\begin{aligned} x_{12} &= a_{11}b_{12} + a_{12}b_{22} \\ x_{13} &= a_{11}b_{13} + a_{12}b_{23} \\ x_{32} &= a_{31}b_{12} + a_{32}b_{22} \\ x_{33} &= a_{31}b_{13} + a_{32}b_{23} \end{aligned}$$

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Exercice: tableaux et fonctions

Operations sur les matrices:

8. Ecrire la fonction main pour tester les différentes opérations.

```
public static void main(String[] args) {  
    int[][] matrice= {{1,2},{3,4}};  
    int[][] matrice2= {{5,6,5},{7,8,7}};  
    int[][] transpose=transpose(matrice2);  
    System.out.println("Original M1");  
    afficherMatrice(matrice);  
    System.out.println("Original M2");  
    afficherMatrice(matrice2);  
    System.out.println("Transposé M1");  
    afficherMatrice(transpose);  
    ..  
}
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Exercice: tableaux et fonctions

```
public static void main(String[] args) {  
    System.out.println("Addition de M1 et M2");  
    if(memeTaille(matrice,matrice2)==true)  
        afficherMatrice(addition(matrice,matrice2));  
    else  
        System.out.println("  
            + \"Les 2 matrices doivent avoir la même taille pour l'addition\"");  
  
    System.out.println("Multiplication de M1 et M2");  
    if(compatibleMultiplication(matrice,matrice2)==true)  
        afficherMatrice(multiplication(matrice,matrice2));  
    else  
        System.out.println("  
            + \"Les 2 matrices doivent être compatibles pour la multiplication\"");  
}
```

II. SYNTAXE ET ÉLÉMENTS DE BASE DU LANGAGE JAVA

Exercice: tableaux et fonctions

Exercice

1. Ecrire une fonction qui retourne la matrice suivante pour une taille donnée (5 par exemple):

1	1
3	9
5	25
7	49
9	81